Concierto Lead Intelligence

Project Documentation

Internal lead qualification platform for the Trianz sales team.

BANT+HS scoring · Claude AI · Bulk processing · Team-wide history

Live URL	`https://concierto-lead-qualification.azurewebsites.net`
Repository	`https://git.trianz.com/fin_strat/concierto-lead-intelligence`
Owner	`Deepak V (AI Engineer Intern)`
Document version	`1.0`
Last updated	`17 June 2026`

0. Background & Context

0.1 What is a "lead", and why qualify it?

A **lead** is any potential customer who could buy from Trianz — typically a company exploring cloud migration, modernisation, data, or AI services. Trianz receives leads from many channels: events, marketing campaigns, partner referrals (AWS, Azure), inbound enquiries, LinkedIn outreach, etc.

But not every lead is worth pursuing. Some have no budget. Some don't have a real decision-maker engaged. Some are years away from buying. Some compete directly with a partner Trianz works with.

Lead qualification is the process of deciding which leads are worth a sales rep's time, and which should be politely declined or nurtured for later.

The challenge: this judgment is normally **gut feel + inconsistent templates**. One rep might say "this is hot!" because the contact replied quickly; another might say "this is dead" because the budget isn't approved yet. Different reps qualify the same lead differently. Trianz then loses time chasing weak leads and misses real opportunities.

0.2 What this app solves

Concierto Lead Intelligence replaces "gut feel" with a deterministic 18-point scoring framework (BANT+HS) plus AI-generated context. Every Trianz rep qualifies leads the same way, and the AI suggests next actions tailored to the lead's specific situation.

Concrete outcomes:

Before	After
Each rep scores leads differently	Consistent BANT+HS scoring across the team
Decisions made on subjective impressions	18-point score grounded in 6 measurable dimensions
No structured "next step" guidance	AI-generated next actions with urgency tags
Outreach emails written ad-hoc	AI-drafted personalised email per lead
50-lead lists triaged manually in Excel	Bulk upload, auto-score 500 leads in seconds
Pipeline visibility scattered across reps' notes	Shared cloud history, all Trianz reps see the same view
Hours to qualify a list	Minutes

0.3 BANT+HS — the framework explained

Classic **BANT** (Budget, Authority, Need, Timeline) has been used in B2B sales for decades. It captures the four dimensions any rep needs to qualify a deal:

- **Budget** — does the prospect have money to spend?
- **Authority** — is the contact a real decision-maker, or just a researcher?
- **Need** — is there a real business problem driving this conversation?
- **Timeline** — when do they intend to buy?

For Trianz, two more dimensions matter heavily:

- **Scope clarity (S)** — does the prospect actually know what they want done? A clear "migrate 60 workloads to AWS by Q4" is very different from "we're thinking about cloud someday".
- **Hyperscaler alignment (H)** — is the prospect already aligned with an AWS / Azure / GCP partner that Trianz co-sells with? Trianz's business model heavily depends on hyperscaler co-sell motions. A lead where AWS or Azure is actively co-selling with us is dramatically more likely to close.

That's why the framework is **BANT+HS** — the standard 4 + the 2 that matter for our specific business.

Each dimension is scored 0–3, totalling out of 18. The total maps to four tiers: **HOT / QUALIFIED / NURTURE / DISQUALIFY** — each with a recommended sales motion.

Full scoring matrix is in Section 5.

0.4 Who uses this app

Role	What they do in the app
Sales rep / SDR	Single-lead qualifies every new contact, gets AI next-actions, drafts outreach emails
Account Executive (AE)	Reviews HOT leads from history before discovery calls; sees AI analyst notes for context
Sales operations	Uses bulk upload for marketing-event lead lists (e.g. AWS re:Invent attendee list of 500 names) and pipeline summary for territory rollups
Sales leadership	Skims the team-wide history to see pipeline depth, HOT lead counts, and where reps are spending time

The app is designed for **internal Trianz use only** — restricted via Microsoft Entra ID to `@trianz.com` accounts.

0.5 Typical user workflow

Single lead scenario — rep just got off a discovery call with Acme Corp's CTO

1. Opens the app → **Single Lead** mode
2. Fills in company, contact, title, industry, revenue band, cloud preference
3. Picks one option for each of the 6 BANT+HS dimensions based on what they learned on the call
4. Clicks **Qualify Lead** → score appears instantly (e.g. 16/18 HOT)
5. AI analysis loads in ~7 seconds:
    - "Strong fit — CTO authority + funded budget + DC exit pressure..."
    - Recommended Trianz offerings: *Concierto Assess, Migrate*
    - Next actions: "Schedule architecture discovery within 48 hrs (IMMEDIATE)..."
    - Data to enrich: "Confirm VMware contract expiry date..."
6. Rep clicks **Draft Email** → personalised outreach email appears, ready to copy into Outlook
7. Rep clicks **Save to History** → lead is stored in shared cloud DB for the team to see

Bulk scenario — sales ops just got a list of 300 conference attendees from marketing

8. Opens the app → **Bulk Upload** mode
9. Downloads the CSV template, fills in 300 rows with what's known (sometimes incomplete)
10. Uploads the CSV → 300 leads are auto-scored in 2 seconds
11. Filters the results: "show me HOT only" → narrows to 12 leads
12. Selects those 12 → clicks **Draft Emails** → AI generates 12 personalised emails in parallel
13. Generates a pipeline summary across all 300 → exec-level paragraph: *"Pipeline contains 12 immediately actionable HOT leads concentrated in Banking and Healthcare; budget signals strong in 80% of HOT leads; H scores indicate strong AWS co-sell alignment..."*
14. Saves selected leads to history → distributes follow-ups to AEs

0.6 The AI's role (Claude Haiku 4.5)

The scoring engine is **fully deterministic** — no AI required, scores are calculated client-side from the user's input. This means consistent, fast, predictable results.

The AI layers on top of that score, doing things the scoring math can't:

15. **Context interpretation** — *"this 16/18 HOT lead has B=2 (budgetable but not funded yet); the rep should focus on getting CFO sign-off before going deep on tech"*
16. **Trianz offerings mapping** — *"based on this lead's profile, recommend Concierto Assess (cloud-readiness audit) + Concierto Migrate (workload migration) but skip Concierto Modernize (not yet relevant)"*
17. **Personalised next actions** — *"Schedule discovery call with Jane (CTO) within 48 hrs to confirm workload inventory and DC exit deadline"* — specific, not generic
18. **Outreach email drafting** — writes the actual email body, customised to the lead's industry, pain context, and tier
19. **Pipeline rollups** — summarises 300 leads into a 4-sentence exec brief

The AI is invoked via Anthropic's tool-use API which guarantees structured JSON output (no parsing errors, no hallucinated formats). System prompts are tightly scoped to Trianz's sales motion.

0.7 Trianz offerings referenced

The AI maps leads to these Trianz product names where relevant:

Offering	When recommended
Concierto Compass	Strategy / advisory — early-stage exploration, scope-defining
Concierto Assess	Cloud-readiness audit, workload discovery
Concierto Migrate	Active workload migration to AWS/Azure
Concierto Manage	Post-migration managed services, ongoing operations
Concierto Modernize	App refactoring, containers, microservices

These are the actual Trianz service lines — the AI recommends them based on lead context, not at random.

0.8 Why this is a POC, not a finished product

The current scope is intentionally narrow:

- Single user (you / a tester) at a time, no live collaboration
- Email-based identity (until Entra ID restriction kicks in fully across both Trianz tenants)
- No CRM integration (yet) — exports to CSV/Excel/PDF instead of pushing to Salesforce
- No notifications (Slack/Teams alerts for HOT leads not yet built)

The goal of the POC: prove the **scoring framework + AI assistance + cloud history** combination delivers real value before Trianz invests in deeper integration. Once validated, the natural next steps are CRM hooks, Slack notifications, and broader rollout to the sales org.

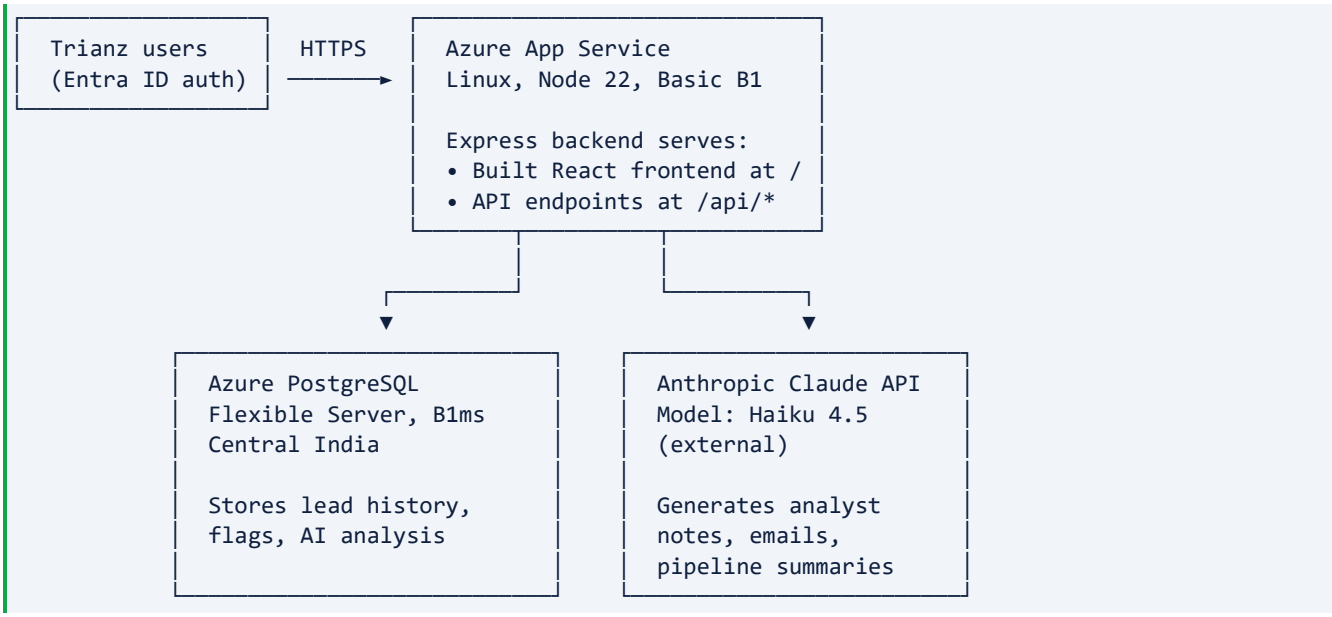
1. Purpose

Sales teams traditionally qualify leads through gut feel and inconsistent templates. This app standardises the process:

20. A rep enters lead data (company, contact, signals)
21. The app **scores** the lead deterministically across 6 dimensions (BANT+HS framework, 18-point scale)
22. **Claude AI** generates analyst notes, recommended Trianz offerings, next actions, and an outreach email draft
23. Results are saved to a shared team history so all reps see the same pipeline view

Outcome: faster qualification, consistent scoring across reps, AI-assisted next steps, no more "I think this lead is hot" subjectivity.

2. Architecture



Hosting region: Central India (low latency for India-based Trianz users)
Identity provider: Microsoft Entra ID, single-tenant (Trianz only)
Estimated cost: ~$25/month (App Service + PostgreSQL)

3. Tech Stack

Layer	Technology	Notes
Frontend	React 18 + TypeScript + Vite	SPA, mobile-responsive
Backend	Node 22 + Express + TypeScript	Single Express app serves frontend + API
Database	PostgreSQL 18 (Azure Flexible Server)	JSONB columns for nested data
AI	Anthropic Claude Haiku 4.5	Tool-use mode for structured output
Auth	Microsoft Entra ID via Azure Easy Auth	Single-tenant, restricted to @trianz.com
File parsing	PapaParse (CSV), SheetJS (Excel)	Client-side, no server-side parsing
PDF export	jsPDF	Client-side rendering
Version control	Git + GitLab (git.trianz.com)	Hosted internally
Deployment	GitLab → Azure External Git → Oryx build → App Service	Push to deploy

4. Features

4.1 Single Lead Qualification

- 6 BANT+HS signals (dropdowns), company info, cloud infrastructure tags
- Real-time score and tier display as user types
- Title vs Authority sanity check (warns if title doesn't match selected Authority level)
- AI-generated analyst notes, recommended offerings, next actions, discovery questions
- Export as PDF, Excel, or CSV
- AI-drafted outreach email with copy-to-clipboard

4.2 Bulk Upload

- CSV or Excel template, up to 500 leads per batch
- Smart column detection (handles `B_Budget Signal`, `Budget Clarity`, `B`, etc.)
- Auto-scoring on upload (deterministic, instant)
- Tier filter pills + free-text search
- Pipeline summary across selected leads (AI generates exec-level rollup)

- Bulk outreach emails for selected or all HOT leads
- Per-row "Save to History" + bulk "Save Leads to History"

4.3 Lead History

- Shared across all Trianz users (stored in Azure PostgreSQL)
- Each entry tagged with `saved_by` (Trianz email)
- Re-open any past lead — restores form, score, AI analysis
- Delete individual entries or clear all
- PDF export from history
- localStorage fallback if database temporarily unavailable
- "Sync local → cloud" button to migrate offline-saved entries

4.4 Mobile-Responsive

- Tested at 375px (phone), 768px (tablet), 1280px+ (desktop)
- Header collapses on narrow screens
- Lead form remains usable on mobile

5. BANT+HS Framework

Six scoring dimensions, each scored 0–3, summed to a maximum of 18 points.

Dim	3	2	1	0
B Budget	Funded	Budgetable	Unfunded	Blocked
A Authority	Decision-maker (CIO/CTO/CFO/VP Infra)	Influencer (Director)	Practitioner (IT Mgr/Architect)	No access
N Need	Critical mandate	Important pain	Nice-to-have	No pain
S Scope	Clear scope	Partial clarity	Undefined	Opposed
T Timeline	Immediate + exit pressure	Near-term (6–12 mo)	Undefined	Deferred
H Hyperscaler	Actively co-selling (AWS/Azure AE engaged)	Aware but passive	No relationship	Different partner

Tier mapping

Tier	Total	Action
HOT	15–18	Fast-track to AE, schedule discovery within 48 hrs
QUALIFIED	11–14	Assign to AE, standard sales motion, discovery within 2 weeks
NURTURE	7–10	Marketing-led engagement, reassess in 60–90 days
DISQUALIFY	≤6	Polite exit, do not invest AE time

Hard rules (forced DISQUALIFY)

These bypass the score and force a DISQUALIFY tier:

- `B = 0` → no funding path
- `A = 0` → contact unresponsive or blocking
- `N = 0` AND `T = 0` → no pain, no urgency

Soft flags (warnings, do not change tier)

- `A = 1` → practitioner contact, escalate to a decision-maker
- `H = 0` → competitor SI engaged, displacement strategy required
- `N = 1` AND `T = 1` → nurture-only flag
- Revenue < $500M → SMB / Partner route flag

6. Project Structure

```
concierto-lead-intelligence/
├── backend/                        Node + Express server
│   ├── src/
│   │   ├── routes/                 /api/qualify, /api/draft-email, /api/pipeline-summary,
/api/history
│   │   ├── services/               Anthropic Claude integration
│   │   ├── migrations/             PostgreSQL schema (auto-run on startup)
│   │   ├── prompts/                System prompts for AI calls
│   │   ├── middleware/             Error handler
│   │   ├── db.ts                   pg connection pool
│   │   └── server.ts               Express setup, security headers, serves frontend in prod
│   ├── .env                        Secrets (gitignored)
│   ├── .env.example                Template
│   └── package.json
│
├── frontend/                       React + Vite SPA
│   └── src/
│       ├── components/
│       │   ├── single/             SingleLeadPage, LeadForm, ResultsPanel
│       │   ├── bulk/               BulkUploadPage, BulkResultsPage
│       │   ├── HistoryPage.tsx
│       │   ├── EmailPrompt.tsx     First-visit Trianz email capture
│       │   ├── LandingPage.tsx
│       │   └── Header.tsx
│       ├── hooks/
│       │   ├── useLeadHistory.ts   Cloud-first with localStorage fallback
│       │   └── useWindowWidth.ts   Responsive breakpoints
│       ├── api/qualify.ts          Server call helpers
│       ├── scoring/                Pure functions: scorer, dims, tiers, signalMap, titleCheck
│       ├── utils/pdfExport.ts      jsPDF rendering
│       └── types/                  TypeScript types
│
├── .gitignore
├── .gitlab-ci.yml                  CI/CD pipeline (currently unused; manual deploys)
├── package.json                    Root orchestration (npm run build, npm start)
├── README.md
├── SETUP.md                        Local dev guide for new contributors
└── DOCUMENTATION.md                This file
```

7. Local Development

Prerequisites: Node.js 20+, code editor (VS Code).

```
# 1. Clone repo
git clone https://git.trianz.com/fin_strat/concierto-lead-intelligence.git
cd concierto-lead-intelligence

# 2. Install dependencies (both folders)
npm run install:all
```

```
# 3. Set up backend/.env (see backend/.env.example)
#    Required: ANTHROPIC_API_KEY, DATABASE_URL
#    Optional: PORT (default 3001), NODE_ENV=development

# 4. Run backend (terminal 1)
npm run dev:backend
# → http://localhost:3001

# 5. Run frontend (terminal 2)
npm run dev:frontend
# → http://localhost:5173
```

Full setup walkthrough is in `SETUP.md` (for new colleagues).

8. Environment Variables

Backend reads from `backend/.env` (local) or Azure App Service Configuration (production).

Variable	Description	Example
`ANTHROPIC_API_KEY`	Claude API key	`sk-ant-...`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pwd@host:5432/postgres?sslmode=require`
`NODE_ENV`	`development` or `production`	`production` (Azure)
`PORT`	Backend port	`3001` (local), `8080` (Azure Linux)
`WEBSITES_PORT`	Azure-specific port forwarding	`8080`
`WEBSITE_NODE_DEFAULT_VERSION`	Node version for App Service	`~22`
`SCM_DO_BUILD_DURING_DEPLOYMENT`	Tell Azure to run `npm run build` on deploy	`true`

Variable	Description	Example
`FRONTEND_ORIGIN`	CORS allow-list (dev only)	`http://localhost:5173`

Never commit `.env` files. `.gitignore` protects them.

9. Database Schema

Single table, auto-created on first startup via migration runner (`backend/src/migrations/`).

```
CREATE TABLE IF NOT EXISTS leads (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Identity / audit
  saved_by        TEXT NOT NULL,
  saved_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),

  -- Form inputs
  company         TEXT NOT NULL,
  contact         TEXT,
  title           TEXT,
  industry        TEXT,
  revenue         TEXT,
  cloud_infra     JSONB DEFAULT '[]'::jsonb,
  vmware_expiry   TEXT,
  pain_context    TEXT,
  notes           TEXT,

  -- BANT+HS scores: {B:"3", A:"3", N:"3", S:"3", T:"3", H:"3"}
  scores          JSONB NOT NULL,

  -- Qualification result
  total_score     INTEGER NOT NULL,
  tier            TEXT NOT NULL,
  flags           JSONB DEFAULT '[]'::jsonb,

  -- Full Claude AI response (narrative, offerings, nextActions, dataToEnrich, discoveryQuestions)
  ai_analysis     JSONB
);

CREATE INDEX idx_leads_saved_at      ON leads (saved_at DESC);
CREATE INDEX idx_leads_saved_by      ON leads (saved_by, saved_at DESC);
CREATE INDEX idx_leads_company_lower ON leads (LOWER(company));
CREATE INDEX idx_leads_tier          ON leads (tier);
```

JSONB columns let us store nested data (scores, flags, AI response) without normalising into multiple tables. Queries can index inside JSONB if needed later.

10. API Reference

All endpoints under `/api/*`. JSON request/response. Rate-limited at 500 requests/min per IP (Anthropic-calling endpoints).

Method	Path	Description	Auth
`GET`	`/api/health`	Health check + DB connectivity status	None
`POST`	`/api/qualify`	Generate AI analysis for a scored lead	Rate-limited
`POST`	`/api/draft-email`	Generate outreach email for a lead	Rate-limited
`POST`	`/api/pipeline-summary`	Cross-lead summary across many leads	Rate-limited
`GET`	`/api/history`	List saved leads (newest first), optional `?email=` filter	None in dev, Entra ID in prod
`POST`	`/api/history`	Save a lead to history	None in dev, Entra ID in prod
`DELETE`	`/api/history/:id`	Delete one lead by UUID	None in dev, Entra ID in prod
`DELETE`	`/api/history?email=x`	Bulk delete by user	None in dev, Entra ID in prod

Example — POST /api/qualify

Request:

```
{
  "form": {
    "company": "Acme Corp",
    "contact": "Jane Doe",
    "title": "CTO",
    "industry": "Banking & Financial Services",
    "revenue": "$1B – $5B",
    "cloudInfra": ["AWS"],
    "scores": { "B":"3", "A":"3", "N":"3", "S":"3", "T":"3", "H":"3" },
    "vmwareExpiry": "2025-12-31",
    "painContext": "DC exit mandate",
    "notes": ""
  },
  "scoreResult": {
    "total": 18,
    "tier": "HOT",
    "flags": []
  }
}
```

Response (Claude tool-use guarantees this shape):

```
{
  "narrative": "Strong fit. CTO authority, funded budget, clear scope...",
```

```
  "recommendedOfferings": ["Concierto Assess", "Concierto Migrate"],
  "nextActions": [
    { "action": "Schedule discovery within 48 hrs", "urgency": "IMMEDIATE", "rationale": "..." }
  ],
  "dataToEnrich": ["Confirm VMware contract expiry"],
  "discoveryQuestions": { "B": null, "A": null, "N": null, "S": null, "T": null, "H": null }
}
```

11. Authentication

Production (Azure)

Azure App Service Easy Auth with Microsoft Entra ID identity provider.

- **Restrict access:** Require authentication
- **Unauthenticated requests:** HTTP 302 redirect → Microsoft sign-in
- **Tenant restriction:** Single-tenant (Trianz Digital Consulting Private Limited)
- **App Registration ID:** `28c5ae88-a64c-4998-8049-7874a5bce86b`

When a user visits the URL:

24. Azure intercepts → redirects to login.microsoftonline.com
25. User signs in with `@trianz.com` account
26. Microsoft validates against the tenant → returns token
27. Azure forwards request to app with `X-MS-CLIENT-PRINCIPAL-NAME` header
28. App reads `/.auth/me` to auto-populate the user's email — skips the manual email prompt

Development (localhost)

No Azure auth. App falls back to a one-time **email prompt modal** that asks for the user's `@trianz.com` email and stores it in localStorage. Honor-system only; no real authentication in dev.

Known limitation

Trianz has **two Azure tenants**:

- Tenant A (`376949d1-...`) — Trianz Digital Consulting Private Limited (where this app is deployed)
- Tenant B (`a27f6b6b-...`) — main Trianz tenant (where most employees exist)

Currently locked to Tenant A only. Users in Tenant B get error AADSTS50020. Resolution requires Trianz IT to create an App Registration in Tenant B (open request as of deployment date).

12. Security

- Secrets stored in Azure App Service Configuration (encrypted at rest), never in code or git
- `.env` files gitignored; `.gitignore` also blocks `glpat-*`, `sk-ant-*`, `*token*`, `*secret*` patterns
- All API endpoints use parameterised SQL queries (immune to SQL injection)
- React escapes user-supplied strings (immune to stored XSS)
- HTTP security headers applied: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, HSTS (prod only)

- `X-Powered-By` header disabled to avoid framework fingerprinting
- Rate limiting on AI endpoints (500 req/min per IP)
- Body size limit: 1 MB
- HTTPS-only enforced on App Service
- Personal Access Token for GitLab → Azure deployment scoped to `read_repository` only

13. Deployment

Initial deploy

29. Code pushed to GitLab repo (`fin_strat/concierto-lead-intelligence`)
30. Azure App Service Deployment Center → External Git → connected to GitLab repo with read-only PAT
31. Azure Oryx builder pulls code on `Sync`, runs `npm install` + `npm run build`, restarts app

Future deploys

32. Push to `main` branch (locally: `git push origin main`)
33. Azure auto-detects via External Git polling (every ~3 min)
34. Oryx rebuilds + restarts
35. New version live in ~5 minutes

Manual sync (force redeploy)

Azure Portal → App Service → Deployment Center → click **Sync**.

Cost breakdown

Resource	Tier	Monthly
App Service Plan	Basic B1 (Linux)	~$13
PostgreSQL Flexible	Burstable B1ms, 32 GiB	~$12
Bandwidth (low POC traffic)	First 100 GB free	$0
Application Insights	Free tier	$0
Anthropic Claude API	Pay-per-use	Separate Anthropic bill
Azure subtotal		**~$25/month**

Pause both services when not in use to drop cost to ~$2/month (storage only). Stop button on each resource's Overview page.

14. Known Limitations & Future Work

Limitations

- **Single-tenant auth only** — most Trianz employees in main tenant can't sign in until IT creates App Registration there
- **No automated CI/CD** — current deploys are manual via External Git Sync, not via `.gitlab-ci.yml` pipeline. Adding CI/CD requires `User Access Administrator` role from Trianz IT (Service Principal creation blocked otherwise)
- **No real-time collaboration** — two users editing the same lead don't see each other's changes live
- **No notification system** — no email/Teams alerts for new HOT leads
- **History dedup** — same company can be saved multiple times if scores differ
- **Lead history capped at 50 entries** in localStorage fallback (cloud has no cap)

Potential next steps

- IT ticket: register app in main Trianz tenant → unlock org-wide access
- Add CRM integration (Salesforce / HubSpot export)
- Slack / Teams webhook notifications for new HOT leads
- Real-time multi-user lead editing
- Lead deduplication suggestions
- Custom domain (`concierto.trianz.com` instead of `*.azurewebsites.net`)
- Production Azure SQL backup automation

15. Maintenance

Update the app

```
# Make code changes locally
git add .
git commit -m "your change description"
git push origin main
# Azure auto-deploys in ~5 minutes
```

Restart the app (no code change)

Azure Portal → concierto-lead-qualification → **Restart**

View live logs

Azure Portal → concierto-lead-qualification → **Log stream**
Or via CLI: `az webapp log tail --resource-group concierto-lead-rg --name concierto-lead-qualification`

Pause the app (save cost)

Azure Portal → concierto-lead-qualification → **Stop**
Also stop PostgreSQL: `concierto-pg` → **Stop**

Resume the app

Same place → **Start**. App is live in ~30 seconds.

Rotate secrets

- Anthropic key: regenerate at https://console.anthropic.com/settings/keys → update `ANTHROPIC_API_KEY` in App Service Configuration + restart
- Postgres password: Portal → concierto-pg → **Reset password** → update `DATABASE_URL` in App Service Configuration + restart
- GitLab PAT: GitLab → Preferences → Access Tokens → revoke + create new → update Deployment Center password field

16. Contacts

Role	Person
Project owner / developer	Deepak V (AI Engineer Intern, Trianz)
Code repo	https://git.trianz.com/fin_strat/concierto-lead-intelligence
Azure subscription	Trianz Digital Consulting Private Limited (`Azure subscription 1`)
AI provider	Anthropic (https://console.anthropic.com)

For issues, questions, or feature requests: contact Deepak directly or open a GitLab issue on the repo.

Last updated: 17 June 2026 · Document version: 1.0